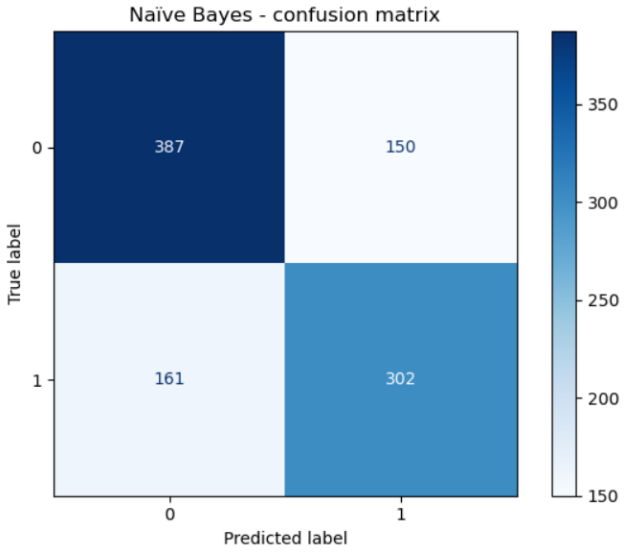
Assignment 3 – Report For the Churn Modelling Dataset

The dataset that was provided by the professor, was on “churn modelling”. We started off with some preprocessing just to be sure that the dataset was clean by; dropping missing rows and removing duplicates which, there were none as well as converting categorical attributes to numerical which in this case were the “Geography” and the “Gender” features.

We then created a **Naïve Baynes** classification on the test data and achieved an accuracy of 68.9%. The result was depicted by our “Confusion Matrix” below.



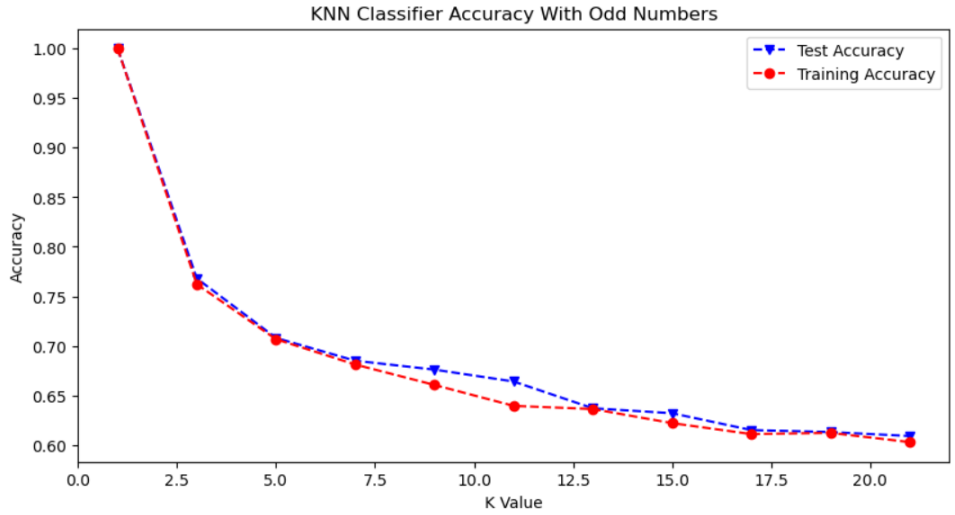
The model was able to predict 387 'True Negatives' and 302 'True Positives' correctly. It predicted 150 'False Positives' and 161 'False Negatives'. The overall accuracy of the model was 68.9%, which is certainly not too bad. This is a normal test ratio since it depicted that our model was not biased towards one class, i.e., either 0 or 1. Furthermore, this low percentage also may indicate that some data may be mis-classified, about 30%.

We then attempted to create a **K-nearest neighbor** (KNN) model. We tested many values of *k* which ranged from 1 to 22. These were the printed results of each *k* value along with the graph:

```
---Trying odd values of k from 1 to 21---
KNN Accuracy achieved with k=1 is: 1.0
KNN Accuracy achieved with k=3 is: 0.7621502209131075
KNN Accuracy achieved with k=5 is: 0.7066764850270005
KNN Accuracy achieved with k=7 is: 0.6811487481590575
KNN Accuracy achieved with k=9 is: 0.6605301914580265
KNN Accuracy achieved with k=11 is: 0.6394207167403043
KNN Accuracy achieved with k=13 is: 0.6362297496318114
KNN Accuracy achieved with k=15 is: 0.6219931271477663
KNN Accuracy achieved with k=17 is: 0.6109474717722141
KNN Accuracy achieved with k=19 is: 0.6121747668139421
KNN Accuracy achieved with k=21 is: 0.602847324496809

---Trying even values of k from 2 to 2---
KNN Accuracy achieved with k=2 is: 0.7641138929798723
KNN Accuracy achieved with k=4 is: 0.7027491408934707
KNN Accuracy achieved with k=6 is: 0.6757486499754541
```

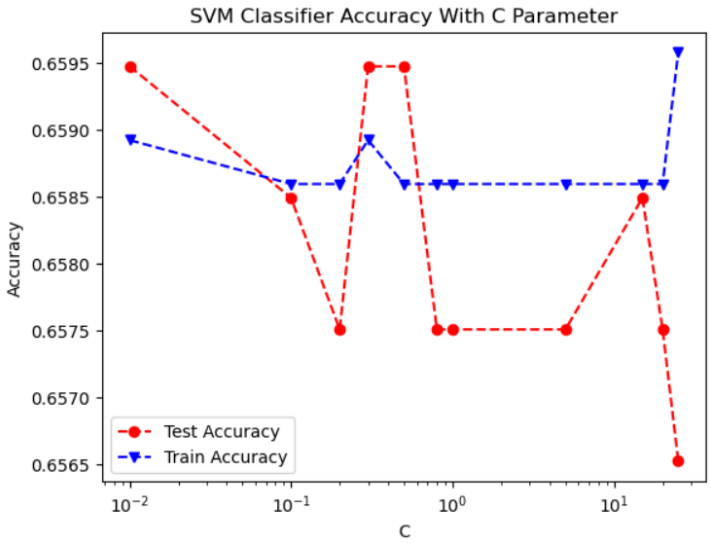
```
KNN Accuracy achieved with k=8 is: 0.6561119293078056
KNN Accuracy achieved with k=10 is: 0.6480117820324006
KNN Accuracy achieved with k=12 is: 0.6325478645066274
KNN Accuracy achieved with k=14 is: 0.6237113402061856
KNN Accuracy achieved with k=16 is: 0.6156111929307806
KNN Accuracy achieved with k=18 is: 0.6094747177221405
KNN Accuracy achieved with k=20 is: 0.6077565046637211
KNN Accuracy achieved with k=22 is: 0.602847324496809
```



The graph shows the different accuracy levels for different values of k. The highest accuracy is seen at when *k* = 2 for even cases. With odd cases, *k* = *3* was the best. At *k* = 1, the model was memorizing the data because it had not seen the data yet and there was very low bias with high variance. At *k* = 21, the model was underfitting the data because it had seen all the data and therefore, there was a lot of bias. Between *k* = 5 and 9, we still had high accuracy, but the performance had dropped. Our test accuracy was very well able to predict the data with a 76.41% accuracy which was very good.

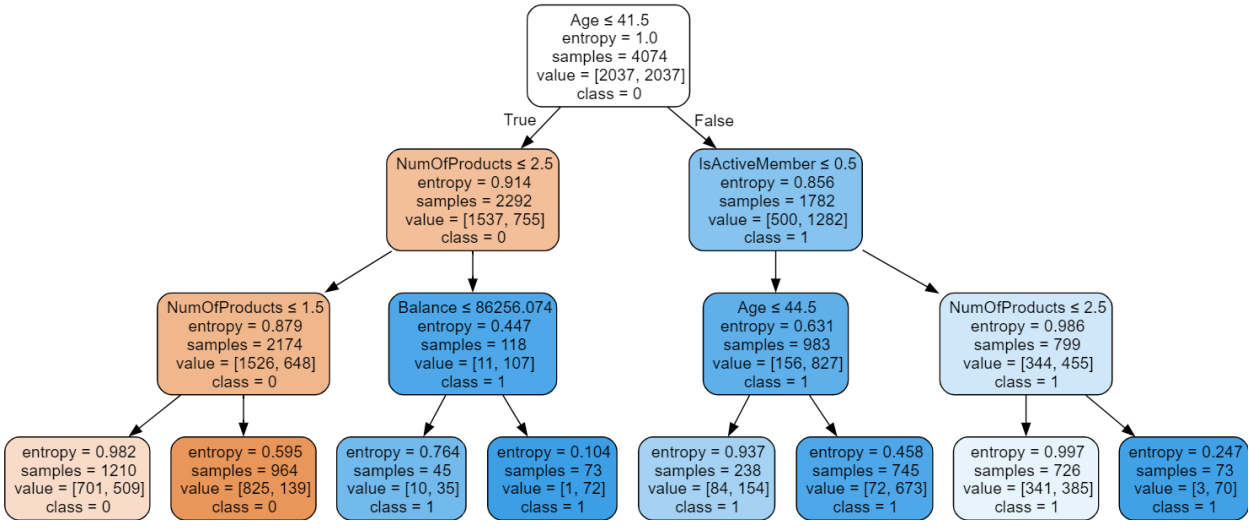
Then, we created a classification for **Support Vector Machine (SVM)**. We tried different values for the C-parameter and got the results along with the graph as below:

```
SVM Accuracy achieved with C=0.01 is: 0.6589198036006547
SVM Accuracy achieved with C=0.1 is: 0.6585924713584288
SVM Accuracy achieved with C=0.2 is: 0.6585924713584288
SVM Accuracy achieved with C=0.3 is: 0.6589198036006547
SVM Accuracy achieved with C=0.5 is: 0.6585924713584288
SVM Accuracy achieved with C=0.8 is: 0.6585924713584288
SVM Accuracy achieved with C=1 is: 0.6585924713584288
SVM Accuracy achieved with C=5 is: 0.6585924713584288
SVM Accuracy achieved with C=15 is: 0.6585924713584288
SVM Accuracy achieved with C=20 is: 0.6585924713584288
SVM Accuracy achieved with C=25 is: 0.6595744680851063
```



The model was stable and accurate for its predicted values with C-parameters below 0.3 with an accuracy of 65.89 percent with a C-value of 0.01. At C-parameter of 15, the model underfits the data having learned all the data values. This may be due to us not properly scaling our data. There was then a sharp decrease in accuracy which may have been due to under fitting as the model had now seen almost all the data values. When the C-parameter went above 10, the model's accuracy started to shoot back up again until it became stable throughout. The takeaway was for us was to have a smaller value for C-parameter to get a stable accuracy.

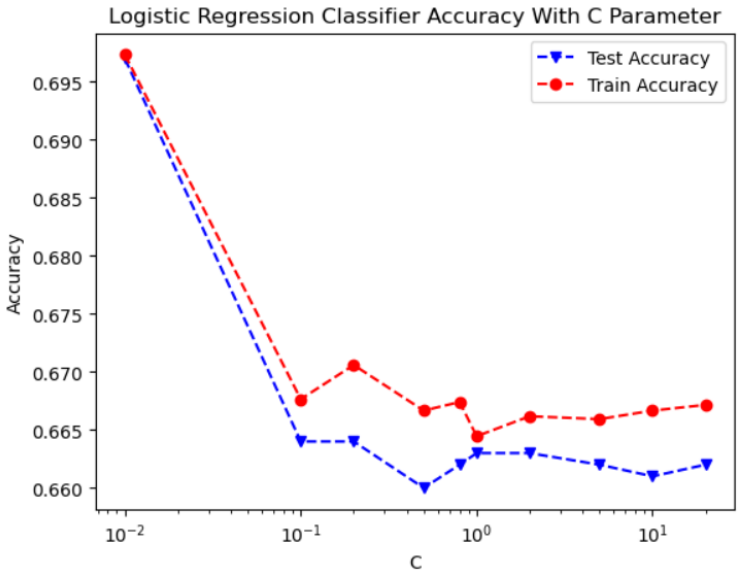
We then created a **Decision Tree** classifier with a maximum depth of 3. We achieved an overall accuracy score of 71.3%. The 'Age' feature was the most important feature in the decision tree model. The mean age was 41.5 (42 round up), and leaving the subscription depended on whether the person was above or below that age. Then, the 'NumberOfProducts' feature showed that those with greater or equal to 2 products are more likely to stay while older customers who aren't active are more likely to leave a subscription. The decision tree that was populated is attached below:



Finally, we created a logistic regression model using different values of C-parameters. The values that we used were: 0.01, 0.1, 0.2, 0.5, 0.8, 1, 2, 5, 10 and 20 and these were the results which we obtained:

```
Logistic Regression Accuracy achieved with C=0.01 is: 0.6973490427098674
Logistic Regression Accuracy achieved with C=0.1 is: 0.667648502700049
Logistic Regression Accuracy achieved with C=0.2 is: 0.6705940108001963
Logistic Regression Accuracy achieved with C=0.5 is: 0.6666666666666666
Logistic Regression Accuracy achieved with C=0.8 is: 0.6674030436917034
Logistic Regression Accuracy achieved with C=1 is: 0.6644575355915562
Logistic Regression Accuracy achieved with C=2 is: 0.6661757486499754
Logistic Regression Accuracy achieved with C=5 is: 0.6659302896416298
Logistic Regression Accuracy achieved with C=10 is: 0.6666666666666666
Logistic Regression Accuracy achieved with C=20 is: 0.6671575846833578
```

The highest accuracy score which we had obtained was 69.73%. We could see that with all the different values of C-parameter tested, the best one was when c = 0.01. It gave a predicted highest value of 69.73 percent accuracy. This meant there was a very strong regularization. The model dipped around the highest point but stabilized with the higher values of C. There was moderate regularization with C-parameters of 0.5-1.5 and weak above values of 3. This showcased that the higher the value of the C-parameter, the weaker the regularization strength and vice versa.

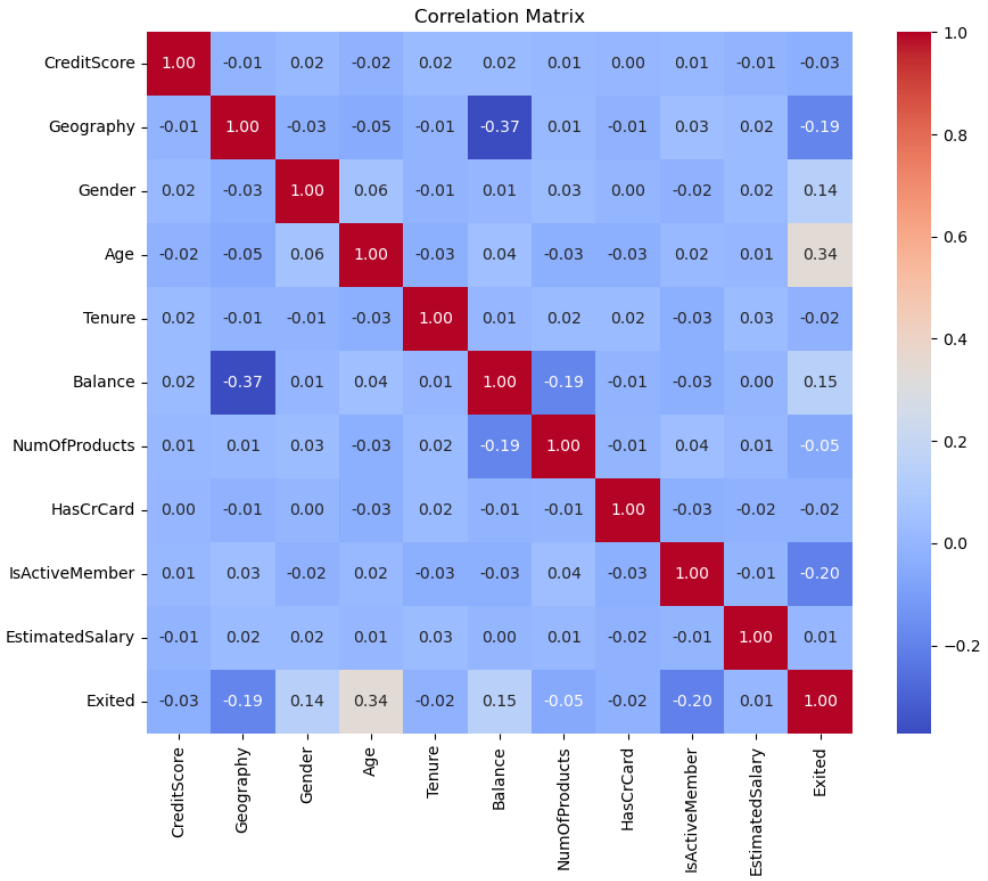


The overall highest accuracy scores for these 5 classification models pulled straight from our code are as below:

Accuracy Table Scores

ML Model	Accuracy on Test Set (Provide accuracy in %)
Naive Bayes	68.89999999999999
KNN	76.41 , with k=3, gave the highest accuracy
SVM	79.63
Logistic Regression	69.73
Decision Tree	71.3

This is what the correlation matrix for the churn modelling dataset looked like:



The closer the number of feature rows with the feature column to either a 1 or -1, the more correlated they were. The closer the value is to 1, the more the correlation they have in a positive direction and the closer to -1, the more correlated they are in a negative direction. A value of 0 meant no correlation. For example, 'CreditScore' showed no correlation to 'Gender' as it had a value of 0.00. On the other hand, 'Age' and 'Exited' had a correlation of 0.29 which meant they were positively correlated, which made sense since older aged people were more likely to leave a subscription since they watched fewer shows, etc., than the younger generation. On the other hand, 'Balance' and 'NumOfProducts' had a correlation of -0.30 which meant that they were negatively correlated, which also made sense since the more products you have, the less balance you have in your account.

Finally, since different split ratios worked best for a particular classification models, we tested each ratio to see which one performed the best and this is what we got:

```
Model: Logistic Regression, Test Ratio: 0.5, Score: 0.6980854197349042
Model: Logistic Regression, Test Ratio: 0.1, Score: 0.6519607843137255
Model: Logistic Regression, Test Ratio: 0.15, Score: 0.6813725490196079
Model: Logistic Regression, Test Ratio: 0.2, Score: 0.6773006134969325
Model: Logistic Regression, Test Ratio: 0.25, Score: 0.6781157998037292
Model: Logistic Regression, Test Ratio: 0.3, Score: 0.6868356500408831
Model: Logistic Regression, Test Ratio: 0.35, Score: 0.7061711079943899
Model: Logistic Regression, Test Ratio: 0.4, Score: 0.7036809815950921
Model: Logistic Regression, Test Ratio: 0.45, Score: 0.7071973827699018
Model: Logistic Regression, Test Ratio: 0.5, Score: 0.6980854197349042
```

```
Model: Decision Tree, Test Ratio: 0.5, Score: 0.6813942071674031
Model: Decision Tree, Test Ratio: 0.1, Score: 0.6911764705882353
Model: Decision Tree, Test Ratio: 0.15, Score: 0.7205882352941176
Model: Decision Tree, Test Ratio: 0.2, Score: 0.694478527607362
Model: Decision Tree, Test Ratio: 0.25, Score: 0.6849852796859667
Model: Decision Tree, Test Ratio: 0.3, Score: 0.6901062959934587
Model: Decision Tree, Test Ratio: 0.35, Score: 0.7033660589060309
Model: Decision Tree, Test Ratio: 0.4, Score: 0.6760736196319018
Model: Decision Tree, Test Ratio: 0.45, Score: 0.6902944383860414
Model: Decision Tree, Test Ratio: 0.5, Score: 0.6926853215513009

Model: K-Nearest Neighbor, Test Ratio: 0.5, Score: 0.7309769268532155
Model: K-Nearest Neighbor, Test Ratio: 0.1, Score: 0.7254901960784313
Model: K-Nearest Neighbor, Test Ratio: 0.15, Score: 0.7401960784313726
Model: K-Nearest Neighbor, Test Ratio: 0.2, Score: 0.7374233128834355
Model: K-Nearest Neighbor, Test Ratio: 0.25, Score: 0.7399411187438666
Model: K-Nearest Neighbor, Test Ratio: 0.3, Score: 0.7350776778413737
Model: K-Nearest Neighbor, Test Ratio: 0.35, Score: 0.7405329593267882
Model: K-Nearest Neighbor, Test Ratio: 0.4, Score: 0.7386503067484662
Model: K-Nearest Neighbor, Test Ratio: 0.45, Score: 0.7306434023991276
Model: K-Nearest Neighbor, Test Ratio: 0.5, Score: 0.7309769268532155

Model: Support Vector Classifier, Test Ratio: 0.5, Score: 0.7677957781050565
Model: Support Vector Classifier, Test Ratio: 0.1, Score: 0.7524509803921569
Model: Support Vector Classifier, Test Ratio: 0.15, Score: 0.7581699346405228
Model: Support Vector Classifier, Test Ratio: 0.2, Score: 0.7619631901840491
Model: Support Vector Classifier, Test Ratio: 0.25, Score: 0.7703631010794897
Model: Support Vector Classifier, Test Ratio: 0.3, Score: 0.7718724448078496
Model: Support Vector Classifier, Test Ratio: 0.35, Score: 0.7692847124824684
Model: Support Vector Classifier, Test Ratio: 0.4, Score: 0.7687116564417178
Model: Support Vector Classifier, Test Ratio: 0.45, Score: 0.7704471101417666
Model: Support Vector Classifier, Test Ratio: 0.5, Score: 0.7677957781050565

Model: Naive Bayes, Test Ratio: 0.5, Score: 0.6794305351006382
Model: Naive Bayes, Test Ratio: 0.1, Score: 0.6666666666666666
Model: Naive Bayes, Test Ratio: 0.15, Score: 0.6928104575163399
Model: Naive Bayes, Test Ratio: 0.2, Score: 0.6797546012269938
Model: Naive Bayes, Test Ratio: 0.25, Score: 0.6800785083415113
Model: Naive Bayes, Test Ratio: 0.3, Score: 0.6802943581357318
Model: Naive Bayes, Test Ratio: 0.35, Score: 0.7033660589060309
Model: Naive Bayes, Test Ratio: 0.4, Score: 0.6938650306748466
Model: Naive Bayes, Test Ratio: 0.45, Score: 0.6875681570338059
Model: Naive Bayes, Test Ratio: 0.5, Score: 0.6794305351006382
```

Summarizing them in a nice table format to get the highest values from each model, we found that a ratio of 0.30 worked the best and we utilized that ratio for future model train and test splits.

Best Model and their Split Ratio Results:

	Model	Test Ratio	Score
12	Decision Tree	0.15	0.720588
26	K-Nearest Neighbor	0.35	0.740533
8	Logistic Regression	0.45	0.707197
46	Naive Bayes	0.35	0.703366
35	Support Vector Classifier	0.30	0.771872